# TEMA 4. Control de errores. Depuración



Desarrollo Web en Entorno Cliente.

# Índice :


- Captura y lanzamiento de errores.
- Depuración.
- Bibliografía.

# Captura y lanzamientos de errores.

Podemos usar el conjunto de palabras claves **try**,**catch**,**throw** y **finally**, para probar un bloque de código en busca de errores y controlarlo.

- **try**, contiene el código a probar.
- **catch**, ejecuta el código en caso de darse alguna excepción o error.
- **throw,** puede generar excepciones o errores personalizados (opcional) .
- **finally**, ejecuta el código independientemente a que se den errores (opcional).

(Ver sintaxis y ejemplos a continuación)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8" >
  <script>
try {
    alert("Hola");
    noExisteFuncion();
    alert ("Adiós");
}
catch(err) {
  alert (err);
}
  </script>
</head>
<body>
</body>
</html>
```

← El siguiente ejemplo devuelve un error al no estar definida la función. El parámetro de error “**err**” , se define como un objeto con dos propiedades : **name** y **message**.

→ El siguiente ejemplo muestra cómo generar un error personalizado con **throw** y el uso de **finally**.

```
<script>
    var n = prompt("Introduzca un número : ");
    try {
        if(n == "")  throw "está vacío";
        if(isNaN(n)) throw "no es un número";
        n = Number(n);
        alert ("Se ha introducido el número: "+ n);
    }
    catch(err) {
        alert ("El valor "+ err);
    }
    finally {
        alert ("Siempre se ejecuta");
    }
</script>
```

# Tipos de errores nativos

| Tipo de error | Descripción | Ejemplo de lanzado |
|---|---|---|
| RangeError | Error de rangos (Ej. valores desorbitados) | var num = 1;<br>num.toPrecision(500); |
| ReferenceError | Error en referencias (Ej. a variables inexistentes) | var x = 0;<br>x = y + 1; |
| SyntaxError | Errores sintácticos (Ej. falta una comilla simple) | eval("alert('Hola)"); |
| TypeError | Errores por tipo (Ej. conversiones no válidas) | var num = 1;<br>num.toUpperCase(); |
| URIError | Errores en direcciones | decodeURI("%%%"); |

# Depuración.

Como ya sabemos, al codificar un programa en JS podemos obtener errores de difícil detección. Para la búsqueda de estos errores existen varios métodos.

## Método  console.log ( )

La mayoría de los navegadores admiten el envío de valores a la ventana de depuración del navegador. Esto permite comprobar si esos valores son los esperados. Sintaxis:  **console.log ( mensaje )**

Ej:

```
var x = 10;
console.log ( x );
```

Pulsamos F12 en el navegador y nos vamos a la pestaña “**Console**”. Aparece el mensaje enviado...

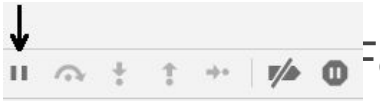
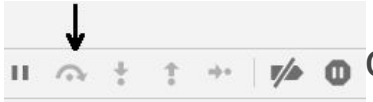
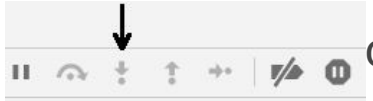
# A través del depurador del navegador.

Por lo general los principales navegadores poseen una serie de herramientas para la depuración.

En este manual vamos a tomar como referencia las herramientas de depuración del Chrome.

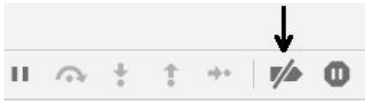
Para depurar un programa en JS con el Chrome podemos seguir los siguientes pasos:

1. Abrimos o ejecutamos el archivo HTML en el navegador.
2. Pulsamos F12 para mostrar las herramientas de desarrollo y depuración.

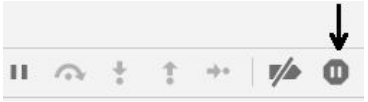
3. Seleccionamos la pestaña “**Sources/Fuentes”**. Podemos ver el código de nuestro archivo.
4. Pulsamos  F8 para entrar en modo pausa en la ejecución.
5. Refrescamos con F5 la página para ejecutar de nuevo, esta vez en modo pausa.
6. Avanzamos con  con F10 para recorrer las instrucciones sin entrar dentro de las funciones.
7. Avanzamos con  con F11 para recorrer las instrucciones una a una incluso dentro de cada función.
8. Si pulsamos  Shift + F11 salimos de la actual función y se ejecuta la siguiente línea de código.

9. Si pulsamos  F9, al igual que con F11 recorremos cada instrucción incluso dentro de las funciones pero añadiendo un comportamiento especial en funciones asíncronas (recorre las líneas de código cronológicamente).
10. Para entrar de nuevo en modo normal (sin depuración), pulsaremos F8 estando en una línea de código de forma pausada.

Podemos crear o eliminar **puntos de interrupción** haciendo click en la línea de código correspondiente, o añadiendo la instrucción "**debugger ;** " , al ejecutar el código hará una pausa permitiéndonos analizar los valores de las variables de dos formas, una colocando el puntero del ratón encima de la variable, y otra, añadiendo la variable en el apartado "**watch**" del depurador.

El siguiente botón activa o desactiva los puntos de interrupción. 

Para activar una pausa siempre que se de un error, con el botón :



13

Paused in debugger

Salida html

Elements Console Sources Network Performance Memory Application Security Audits

Page Filesystem Overrides

top
file://
C:/Users/juan/Documents/JS/Tema%205/pr
p3.html

ej8.html p3.html

```
1  <!DOCTYPE html>
2  <html>
3  <head>
4  </head>
5  <body>
6  <p id="demo"></p>
7  <script>
8          var x = 10;
9          var y = 20;
10         console.log ( x + y );
11         function f1() {
12             document.getElementById("demo").innerHTML = x;
13             document.getElementById("demo").innerHTML = x + 1;
14         }
15         function f2() {
16             let z=0;
17             document.getElementById("demo").innerHTML = x + 2;
18             document.getElementById("demo").innerHTML = x + 3;
19         }
20         f1 ( );
21         f2 ( );
22 </script>
23 </body>
24 </html>
```

Line 22, Column 1

Panel de avance

Punto de interrupción

Alcance de las variables

Debugger paused

Watch

x: 10
y: 20

Variables analizadas

Call Stack

(anonymous) VM4756:1

Scope

Global Window

Breakpoints

p3.html:12
document.getElementById("demo").innerHT…

XHR/fetch Breakpoints

DOM Breakpoints

Global Listeners

Event Listener Breakpoints

Console

top Filter Default levels

30 p3.html:10

Salida consola

NOTA: Para desactivar la entrada del depurador en archivos anónimos (VMxx) ir a Configuración del depurador / Lista de ignorados / Activar casilla “Secuen..Anónimas

# Bibliografía


- LibrosWeb
- W3Schools
- Developer Mozilla Docs